Занятие 3: Проектирование HTTP API

Ресурсы, методы, параметры, статусы

Слайд 1: API как контракт

Что такое API?

- Application Programming Interface
- Контракт между клиентом и сервером
- Определяет доступные операции и формат данных
- Должен быть стабильным и предсказуемым

Ресурс

- Сущность с которой можно взаимодействовать
- Идентифицируется URL
- Примеры: /users, /posts, /orders

Endpoint

- Конкретный URL для операции
- Примеры: GET /users, POST /users, GET /users/123

Маршрутизация

- Сопоставление URL с обработчиком
- Параметры пути и query-параметры

Слайд 2: HTTP-методы и их назначение

Метод	Назначение	Idempotent	Безопасный
GET	Чтение ресурса	Да	Да
POST	Создание ресурса	Нет	Нет
PUT	Полное обновление	Да	Нет
PATCH	Частичное обновление	Нет	Нет
DELETE	Удаление ресурса	Да	Нет

Idempotent - повторный запрос даёт тот же результат
Безопасный - не изменяет состояние сервера

Слайд 3: GET - Получение ресурсов

Использование:

- Получение списка ресурсов
- Получение одного ресурса
- Фильтрация и сортировка

Примеры:

```
GET /users
GET /users/123
GET /users?age_gt=18&sort=name
GET /posts?category=tech&page=1&limit=10
```

Характеристики:

- Idempotent
- Безопасный
- Кэшируется
- Данные в URL (query-параметры)

Слайд 4: POST - Создание ресурса

Использование:

- Создание нового ресурса
- Выполнение операции
- Загрузка файла

Примеры:

```
POST /users
POST /posts
POST /auth/login
POST /upload
```

Тело запроса:

```
{
  "name": "John",
  "email": "john@example.com"
}
```

Характеристики:

- Не idempotent
- Не безопасный
- Обычно возвращает 201 Created
- Данные в теле запроса

Слайд 5: PUT - Полное обновление

Использование:

- Полная замена ресурса
- Создание ресурса (если не существует)

Примеры:

```
PUT /users/123
PUT /posts/456
```

Тело запроса:

```
{
  "id": 123,
  "name": "John Smith",
  "email": "john.smith@example.com",
  "age": 31
}
```

Характеристики:

- Idempotent
- Не безопасный
- Требует все поля
- Отсутствующие поля = null/default

Слайд 6: PATCH - Частичное обновление

Использование:

- Изменение части ресурса
- Обновление отдельных полей

Примеры:

```
PATCH /users/123
PATCH /posts/456/publish
```

Тело запроса:

```
{
  "email": "newemail@example.com"
}
```

Характеристики:

- Не idempotent
- Не безопасный
- Только изменяемые поля
- Более гибкий чем PUT

Слайд 7: DELETE - Удаление ресурса

Использование:

- Удаление ресурса

Примеры:

```
DELETE /users/123
DELETE /posts/456
```

Характеристики:

- Idempotent
- Не безопасный
- Обычно возвращает 204 No Content
- Может возвращать 200 с удалённым объектом

Слайд 8: Параметры пути

Path Parameters

- Часть URL
- Используются для идентификации ресурса

Примеры:

```
GET /users/123
GET /posts/456/comments/789
```

Обозначение в документации:

```
GET /users/{id}
GET /posts/{post_id}/comments/{comment_id}
```

Правила:

- Обязательные
- Иерархические
- Используются для идентификаторов

Слайд 9: Query-параметры

Query Parameters

- После ? в URL
- Используются для фильтрации, сортировки, пагинации

Примеры:

```
GET /users?age_gt=18&sort=name
GET /posts?page=1&limit=10
GET /items?search=laptop&category=electronics
```

Правила:

- Опциональные
- Множественные значения
- Кодируются в URL

Слайд 10: Тело запроса

Когда используется:

- POST, PUT, PATCH
- Загрузка файлов

Форматы:

- JSON (наиболее распространённый)
- XML
- Form data
- Multipart (файлы)

Пример JSON:

```
{
  "name": "John",
  "email": "john@example.com",
  "age": 30,
  "address": {
    "city": "Moscow",
    "street": "Main St"
  }
}
```

Слайд 11: Структура JSON-ответа

Успешный ответ:

```
{
  "id": 123,
  "name": "John",
  "email": "john@example.com",
  "created_at": "2024-01-01T00:00:00Z"
}
```

Список:

```
{
  "users": [
    {"id": 1, "name": "John"},
    {"id": 2, "name": "Jane"}
  ],
  "total": 2,
  "page": 1,
  "pages": 1
}
```

Пагинация:

```
{
  "items": [...],
  "total": 100,
  "page": 1,
  "page_size": 10,
  "pages": 10
}
```

Слайд 12: Статусы успешных операций

Код	Значение	Использование
200 OK	Успех	GET, PUT, PATCH
201 Created	Создан	POST
202 Accepted	Принят	Асинхронные операции
204 No Content	Без тела	DELETE

Примеры:

```
GET /users/123 → 200 OK
POST /users → 201 Created
DELETE /users/123 → 204 No Content
```

Слайд 13: Статусы ошибок клиента

Код	Значение	Использование
400 Bad Request	Некорректный запрос	Ошибка валидации
401 Unauthorized	Требуется авторизация	Нет токена
403 Forbidden	Нет прав	Есть токен, но нет доступа
404 Not Found	Не найден	Ресурс не существует
409 Conflict	Конфликт	Дубликат, нарушение ограничений
422 Unprocessable Entity	Невозможно обработать	Ошибка бизнес-логики

Примеры:

```
POST /users с неверным email → 400 Bad Request
GET /admin без токена → 401 Unauthorized
GET /users/999 → 404 Not Found
```

Слайд 14: Статусы ошибок сервера

Код	Значение	Использование
500 Internal Server Error	Внутренняя ошибка	Ошибка в коде
502 Bad Gateway	Ошибка шлюза	Прокси недоступен
503 Service Unavailable	Недоступен	Перегрузка
504 Gateway Timeout	Тайм-аут	Долгий запрос

Когда использовать:

- 500 - неожиданные ошибки
- 502/504 - проблемы с зависимостями
- 503 - плановое обслуживание

Слайд 15: Идемпотентность

Определение:

- Повторный запрос даёт тот же результат
- Важно для повторных попыток

Идемпотентные методы:

- GET, PUT, DELETE

Не идемпотентные:

- POST, PATCH

Пример:

```
GET /users/123 → всегда один и тот же пользователь
POST /users → создаёт нового пользователя каждый раз
DELETE /users/123 → первый раз удаляет, последующие - 404
```

Слайд 16: Пагинация

Зачем нужна:

- Ограничение размера ответа
- Уменьшение нагрузки
- Улучшение UX

Способы:

- Offset/Limit: `?page=1&limit=10`
- Cursor-based: `?cursor=abc123`
- Keyset: `?last_id=123`

Пример:

```
GET /users?page=1&limit=10

{
  "items": [...],
  "total": 100,
  "page": 1,
  "page_size": 10,
  "pages": 10
}
```

Слайд 17: Практическое задание

Задание 1: Проектирование API
Спроектируйте API для блога:

- Посты (создание, чтение, обновление, удаление)
- Комментарии (создание, чтение)
- Теги (создание, чтение)

Задание 2: Выбор методов
Для каждой операции выберите метод:

- Создать пост
- Получить список постов
- Получить один пост
- Обновить пост
- Удалить пост
- Добавить комментарий

Задание 3: Статусы
Выберите статус-код для:

- Успешное создание
- Ресурс не найден
- Неверные данные
- Успешное удаление

Слайд 18: Вопрос для собеседования

Вопрос: Предложите контракт создания и получения ресурса и объясните выбор методов и статусов.

Ожидаемый ответ:

```
POST /users
Content-Type: application/json

{
  "name": "John",
  "email": "john@example.com"
}

→ 201 Created
{
  "id": 123,
  "name": "John",
  "email": "john@example.com",
  "created_at": "2024-01-01T00:00:00Z"
}

GET /users/123

→ 200 OK
{
  "id": 123,
  "name": "John",
  "email": "john@example.com",
  "created_at": "2024-01-01T00:00:00Z"
}
```

Обоснование:

- POST для создания (не идемпотентный)